# Raft

*Краткое руководство по понятному алгоритму консенсуса*

## Что решает Raft

Raft нужен группе серверов, которые должны вести себя как одна надёжная система. Он поддерживает одинаковый журнал команд на всех узлах, переживает остановку части серверов и не допускает, чтобы разные узлы применили разные команды в одной позиции журнала.

Типичный кластер состоит из пяти серверов и продолжает работать при отказе любых двух, если оставшиеся три могут связаться друг с другом. Модель отказов невизантийская: сервер может остановиться, потерять связь и позднее восстановиться, но не действует злонамеренно.

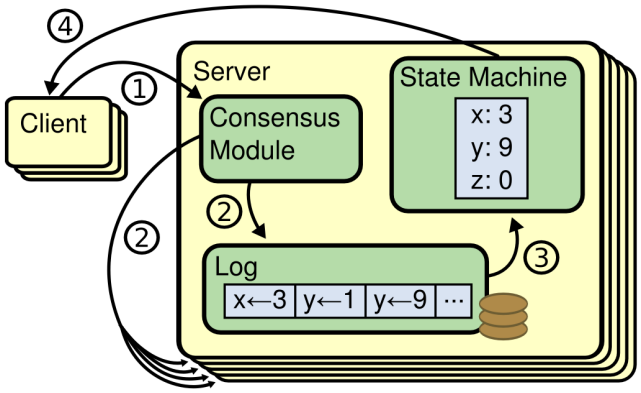



**Figure 1:** Replicated state machine architecture. The consensus algorithm manages a replicated log containing state machine commands from clients. The state machines process identical sequences of commands from the logs, so they produce the same outputs.

*Архитектура реплицированного конечного автомата. Рисунок сохранён в оригинале.*

**Главная идея: сначала выбрать одного лидера, затем проводить через него все изменения журнала. Это делает поток данных односторонним и заметно упрощает рассуждения о системе.**

# 1. Роли, эпохи и выбор лидера

Каждый сервер в любой момент находится в одном из трёх состояний: ведомый, кандидат или лидер. В нормальном режиме один сервер является лидером, остальные ведомыми. Кандидат появляется только на время выборов.

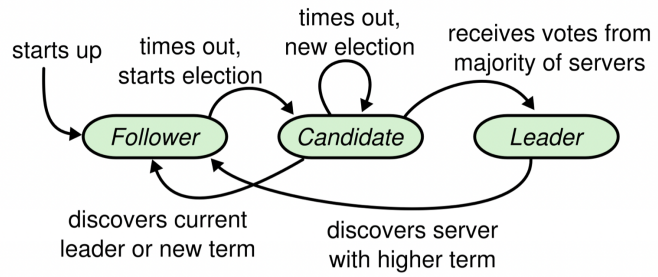



**Figure 4:** Server states. Followers only respond to requests from other servers. If a follower receives no communication, it becomes a candidate and initiates an election. A candidate that receives votes from a majority of the full cluster becomes the new leader. Leaders typically operate until they fail.

*Переходы между состояниями Raft. Рисунок сохранён в оригинале.*

Время разделено на последовательно пронумерованные эпохи. Каждая эпоха начинается с выборов. Если кандидат получает голоса большинства серверов, он становится лидером до конца эпохи. В одной эпохе не может быть двух лидеров, получивших большинство.

## Как проходят выборы

- Лидер периодически отправляет ведомым пустые запросы **AppendEntries**, которые работают как heartbeat-сообщения.
- Если ведомый слишком долго не получает сообщений, он увеличивает номер эпохи, становится кандидатом, голосует за себя и отправляет **RequestVote** другим серверам.
- Кандидат побеждает после получения большинства голосов. При получении сообщения от законного лидера с не меньшей эпохой он снова становится ведомым.
- Таймаут выборов выбирается случайно. Поэтому серверы редко становятся кандидатами одновременно, а разделившиеся голоса быстро разрешаются повторными выборами.

**Безопасность не зависит от скорости сети или часов. Временные параметры влияют только на доступность и скорость выбора нового лидера.**

# 2. Репликация журнала

Клиент отправляет команду лидеру. Лидер добавляет её в свой журнал и параллельно рассылает ведомым через AppendEntries. Когда запись сохранена большинством серверов, лидер считает её зафиксированной, применяет к своему конечному автомату и сообщает результат клиенту.

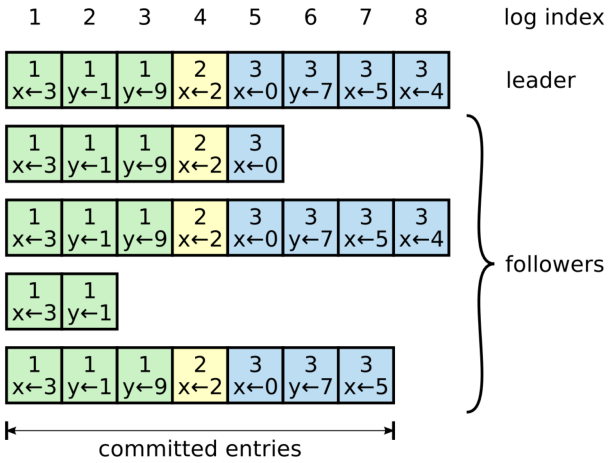



**Figure 6:** Logs are composed of entries, which are numbered sequentially. Each entry contains the term in which it was created (the number in each box) and a command for the state machine. An entry is considered *committed* if it is safe for that entry to be applied to state machines.

*Записи журнала содержат команду, индекс и номер эпохи. Рисунок сохранён в оригинале.*

## Два свойства согласования журналов

- Если две записи имеют одинаковые индекс и номер эпохи, в них хранится одна и та же команда.
- Если записи с одинаковыми индексом и эпохой присутствуют в двух журналах, все предыдущие записи этих журналов также совпадают.

Перед новыми записями лидер передаёт индекс и эпоху предыдущей записи. Ведомый принимает продолжение только при совпадении. Если журналы расходятся, лидер уменьшает **nextIndex**, находит последнюю общую запись, удаляет конфликтующий хвост ведомого и дописывает правильные записи.

**Лидер никогда не перезаписывает собственные записи. После успешной синхронизации журнал ведомого совпадает с журналом лидера и остаётся таким до конца эпохи.**

# 3. Почему Raft безопасен

Одна только репликация недостаточна: сервер с устаревшим журналом мог бы стать лидером и затереть уже зафиксированные команды. Поэтому Raft ограничивает право кандидата на победу. Сервер голосует за кандидата только тогда, когда журнал кандидата не менее актуален, чем его собственный.

Актуальность сравнивается по последней записи: сначала по номеру эпохи, затем, при равных эпохах, по длине журнала. Поскольку для победы необходимо большинство, новый лидер обязательно пересекается хотя бы с одним сервером, на котором хранятся зафиксированные записи.

## Пять ключевых гарантий

| | |
|---|---|
| **Безопасность выборов** | В одной эпохе избирается не более одного лидера. |
| **Только добавление у лидера** | Лидер не удаляет и не перезаписывает записи собственного журнала. |
| **Соответствие журналов** | Одинаковые индекс и эпоха означают одинаковую запись и одинаковый предшествующий префикс. |
| **Полнота лидера** | Каждый будущий лидер содержит все записи, зафиксированные в предыдущих эпохах. |
| **Безопасность автомата** | Два сервера никогда не применят разные команды по одному индексу. |

## Важная тонкость

Лидер напрямую фиксирует подсчётом реплик только записи своей текущей эпохи. Когда такая запись зафиксирована, все предшествующие записи считаются зафиксированными косвенно. Это предотвращает ситуацию, в которой запись старой эпохи временно лежит на большинстве, но затем всё же заменяется будущим лидером.

**Итоговая цепочка рассуждений: ограничение голосования сохраняет зафиксированные записи у будущих лидеров; согласование журналов сохраняет их позиции; применение записей строго по порядку даёт одинаковое состояние на всех узлах.**

# 4. Изменение состава и сокращение журнала

Нельзя безопасно переключить все серверы со старой конфигурации на новую одним шагом: часть узлов может уже считать новой одну конфигурацию, а часть всё ещё использовать старую, из-за чего образуются два независимых большинства.

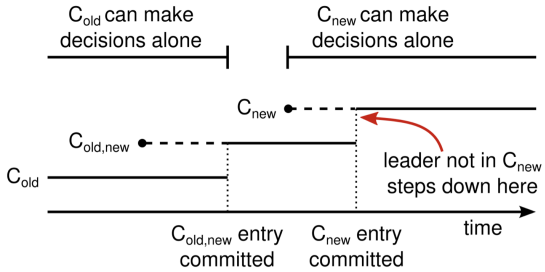



**Figure 11:** Timeline for a configuration change. Dashed lines show configuration entries that have been created but not committed, and solid lines show the latest committed configuration entry. The leader first creates the $C_{old,new}$ configuration entry in its log and commits it to $C_{old,new}$ (a majority of $C_{old}$ and a majority of $C_{new}$). Then it creates the $C_{new}$ entry and commits it to a majority of $C_{new}$. There is no point in time in which $C_{old}$ and $C_{new}$ can both make decisions independently.

*Переход через совместный консенсус. Рисунок сохранён в оригинале.*

Raft использует промежуточную совместную конфигурацию. В ней записи рассылаются всем серверам старого и нового состава, а для выборов и фиксации требуется отдельное большинство в каждой конфигурации. После фиксации совместной конфигурации лидер фиксирует окончательную новую конфигурацию.

## Практические детали

- Новые серверы сначала подключаются без права голоса и догоняют журнал, чтобы не снижать доступность.
- Если лидер не входит в новый состав, он отказывается от роли после фиксации новой конфигурации.
- Удалённые серверы не должны провоцировать бесконечные выборы: запросы RequestVote игнорируются, пока сервер недавно слышал действующего лидера.

## Компактация журнала

Журнал не может расти бесконечно. Сервер периодически сохраняет снимок состояния и удаляет уже учтённый префикс журнала. В снимке остаются индекс и эпоха последней включённой записи, а также текущая конфигурация кластера. Если ведомый слишком отстал, лидер передаёт ему снимок через InstallSnapshot.

# 5. Клиенты и эксплуатация

Клиенты отправляют запросы лидеру. Если они попали на ведомый сервер, тот возвращает адрес известного лидера. При сбое лидера запрос завершается по таймауту, после чего клиент повторяет попытку через другой сервер.

Повтор запроса может привести к повторному выполнению команды. Поэтому клиент добавляет уникальный порядковый номер, а конечный автомат хранит последний обработанный номер и ответ для каждого клиента. Повторная команда получает сохранённый ответ без повторного исполнения.

Для линеаризуемого чтения лидер должен убедиться, что он всё ещё лидер и знает актуальную точку фиксации. В начале эпохи он фиксирует пустую no-op запись, а перед ответом на чтение подтверждает связь с большинством через heartbeat-сообщения.

## Выбор временных параметров

Raft должен удовлетворять практическому соотношению:

**время рассылки << таймаут выборов << среднее время между сбоями**

Слишком короткий таймаут вызывает лишние выборы, слишком длинный увеличивает простой после сбоя лидера. В статье в качестве консервативного ориентира предлагается диапазон 150-300 мс, но реальное значение зависит от сети, дисков и нагрузки.

## Raft в одном абзаце

**Большинство выбирает лидера. Лидер единолично упорядочивает команды, реплицирует их на большинство и сообщает ведомым, что можно применять. Ограничение голосования не допускает лидера без уже зафиксированных записей. При расхождении лидер приводит журналы ведомых к своему, а совместный консенсус позволяет безопасно менять состав кластера.**

Основано на статье Diego Ongaro и John Ousterhout «In Search of an Understandable Consensus Algorithm (Extended Version)». Это сокращённое изложение; для доказательств, экспериментов и деталей протокола следует обращаться к полной версии перевода.